ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ - ΕΜΠ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ & ΜΗΧ. ΥΠΟΛΟΓΙΣΤΩΝ
Τομέας Επικοινωνιών, Ηλεκτρονικής & Συστημάτων Πληροφορικής
Εργαστήριο Διαχείρισης & Βελτίστου Σχεδιασμού Δικτύων Τηλεματικής - NETMODE

ΣΥΣΤΗΜΑΤΑ ΑΝΑΜΟΝΗΣ
(3η άσκηση)

Lab3

6ο Εξάμηνο
Μάρτιος 2026

netmode
Network Management & Optimal Design Laboratory

Γενικά - Προσομοίωση

Δύο κατηγορίες προσομοίωσης

- **Terminating simulation**
- **Non-terminating simulation**

Στασιμότητα και transient

Transient είναι η αρχική μεταβατική περίοδος της προσομοίωσης, όπου τα αποτελέσματα επηρεάζονται από την αρχική κατάσταση

Στασιμότητα είναι το μόνιμο καθεστώς λειτουργίας, όπου οι πιθανότητες καταστάσεων και οι μέσες μετρικές σταθεροποιούνται (για αυτό αγνοούμε τα πρώτα B βήματα σαν αρχικοποίηση)

Για non-terminating simulations πρέπει να ξέρουμε αν το σύστημα γίνεται **stationary (**η στατιστική συμπεριφορά του συστήματος δεν αλλάζει με το πέρασμα του χρόνου)

transient = κατάσταση πριν συγκλίνει
stationary = αφού συγκλίνει στις Pi

Σε Markov queueing μοντέλα, με σταθερό σύστημα, μετά από κάποιο διάστημα το σύστημα θα συγκλίνει σε **stationary regime**, όπως:

- σε M/M/1 με $\rho > 1$ το σύστημα είναι ασταθές,
- σε M/M/1 με $\rho < 1$ γίνεται stationary μετά από transient

M/M/1/10

Θεωρία και λογική προσομοίωσης

Queueing model με αφίξεις Poisson, εκθετική εξυπηρέτηση, έναν server και συνολική χωρητικότητα 10 πελάτες

Poisson arrivals | Exp service | 1 server | Capacity = 10

Ροή συστήματος

Αφίξεις λ → **Σύστημα $0 \le n \le 10$** → **Αναχωρήσεις μ**

Αν n = 10, η νέα άφιξη απορρίπτεται
Αυτό είναι το σημείο που κάνει το μοντέλο ουσιαστικά διαφορετικό από το κλασικό M/M/1

Τι σημαίνει ο συμβολισμός M/M/1/10

M	M	1	10
Markovian arrivals	Markovian service	Ένας εξυπηρετητής	Συνολική χωρητικότητα
Οι αφίξεις είναι Poisson με ρυθμό λ	Οι χρόνοι εξυπηρέτησης είναι εκθετικοί με μέση τιμή 1/μ	Κάθε στιγμή εξυπηρετείται το πολύ ένας πελάτης	Μέσα στο σύστημα χωράνε μέχρι 10 πελάτες

Κρίσιμες παραδοχές

- FCFS πειθαρχία ουράς, εκτός αν οριστεί αλλιώς.
- Ανεξαρτησία μεταξύ αφίξεων και χρόνων εξυπηρέτησης
- Όταν το σύστημα γεμίσει, οι νέες αφίξεις χάνονται

Ερμηνεία για simulation

- Το state space είναι πεπερασμένο: n = 0,1,2,...,10.
- Δεν είναι άπειρη ουρά, αλλά πρέπει να μετράμε blocking
- Το λ σε σχέση με το μ καθορίζει συμφόρηση και losses

Πώς γίνεται η προσομοίωση

1. Εχουμε interarrival χρόνους $A \sim Exp(\lambda)$ και χρονους εξυπηρετησης $S \sim Exp(\mu)$
2. Διατηρούμε την κατάσταση $n(t) \in \{0,\ldots,10\}$ και το επόμενο event
3. Σε άφιξη: αν $n < 10$ δεχομαστε άφιξη, αλλιώς γινεται blocked και χάνεται
4. Σε departure: μειώνεται το n και αν υπάρχει ουρά ξεκινά αμέσως ο επόμενος

Τι μετράμε

- Blocking probability = blocked / total arrivals
- Μέσο πλήθος πελατων στο συστημα L και μέση παραμονή W
- Server utilization και effective throughput
- Σύγκριση θεωρίας vs simulation για validation

Best practice: βάζουμε περίοδος αρχικοποίησης συστήματος, μεγάλη διάρκεια προσομοίωσης και πολλαπλές επαναλήψεις

Εκφώνηση Ασκησης Προσομοίωση συστήματος M/M/1/10 1/3

Θα μελετήσετε, με τη μέθοδο της προσομοίωσης σε ένα σύστημα M/M/1/10, δηλαδή ένα σύστημα με έναν εξυπηρετητή και μέγιστη χωρητικότητα 10 πελάτες. Οι αφίξεις ακολουθούν κατανομή Poisson με μέσο ρυθμό λ πελάτες/min, ενώ οι χρόνοι εξυπηρέτησης είναι εκθετικοί με μέσο ρυθμό εξυπηρέτησης μ=5 πελάτες/min.

Χρησιμοποιώντας τη γλώσσα προγραμματισμού της επιλογής σας, και χωρίς να χρησιμοποιήσετε έτοιμες βιβλιοθήκες προσομοίωσης, να αναπτύξετε πρόγραμμα προσομοίωσης που να υπολογίζει:

- τις εργοδικές πιθανότητες των καταστάσεων του συστήματος,
- την πιθανότητα απόρριψης πελάτη από το σύστημα,
- τον μέσο αριθμό πελατών στο σύστημα,
- τον μέσο χρόνο παραμονής ενός πελάτη στο σύστημα.

Η προσομοίωση θα εκτελείται για προκαθορισμένο συνολικό αριθμό μεταβάσεων *T*. Για τη μελέτη της επίδρασης της αρχικής μεταβατικής περιόδου (**transient period**), τα παραπάνω μεγέθη θα πρέπει να υπολογιστούν:

- μία φορά χρησιμοποιώντας όλες τις μεταβάσεις,
- και μία φορά αγνοώντας τις πρώτες *B* μεταβάσεις.

Να χρησιμοποιήσετε:

- T=100000 και B=5000
- T=1000000 και B=20000
- T=10000000 και B=120000

Σε όλες τις εκτελέσεις της προσομοίωσης θα πρέπει να χρησιμοποιηθεί η ίδια αρχική τιμή seed της γεννήτριας τυχαίων αριθμών, ώστε να παράγονται οι ίδιες ακολουθίες τυχαίων αριθμών και να είναι δυνατή η σωστή σύγκριση των αποτελεσμάτων για διαφορετικές τιμές του λ. Για παράδειγμα, μπορείτε να χρησιμοποιήσετε εντολή της μορφής:

- rand("seed",1)
- ή την αντίστοιχη εντολή της γλώσσας προγραμματισμού που επιλέγετε.

Εκφώνηση Ασκησης 3
2/3

(1) Έλεγχος ορθής λειτουργίας της προσομοίωσης

- Για σκοπούς debugging, να παραγάγετε λεπτομερές trace για τις πρώτες 30 μεταβάσεις της κατάστασης του συστήματος. Για κάθε μετάβαση να καταγράφετε:
- την τρέχουσα κατάσταση του συστήματος,
- το είδος της επόμενης μετάβασης (άφιξη ή αναχώρηση),
- τον συνολικό αριθμό αφίξεων που έχουν συμβεί μέχρι εκείνη τη στιγμή,
- τη νέα κατάσταση του συστήματος μετά τη μετάβαση.
- Αφού επαληθεύσετε ότι η προσομοίωση λειτουργεί σωστά, να απενεργοποιήσετε τον κώδικα που αφορά το debugging και να συνεχίσετε με τις κανονικές εκτελέσεις.

(2) Εκτέλεση της προσομοίωσης για διαφορετικές τιμές του ρυθμού άφιξης

- Να εκτελέσετε την προσομοίωσή σας για τις τιμές: $\lambda=\{1,5,10\}$
- και για κάθε τιμή του λ να υπολογίσετε τα ζητούμενα μεγέθη όπως ζητούνται στην ερώτηση (1):
- με χρήση όλων των μεταβάσεων,
- και με αγνόηση των πρώτων B μεταβάσεων ως transient period.
- Στη συνέχεια, να παραστήσετε γραφικά:
- τις τελικές εργοδικές πιθανότητες των καταστάσεων του συστήματος,
- την εξέλιξη του εκτιμώμενου μέσου αριθμού πελατών στο σύστημα ως προς τον αριθμό μεταβάσεων, για καθεμία από τις τιμές του λ.

Εκφώνηση Ασκησης 3
3/3

(3) Σχολιασμός των αποτελεσμάτων

- Με βάση τα αποτελέσματά σας, να σχολιάσετε:
- πώς επηρεάζει η αφαίρεση της αρχικής μεταβατικής περιόδου τις εκτιμήσεις των μεγεθών,
- πώς μεταβάλλεται η συμπεριφορά του συστήματος καθώς αυξάνεται η τιμή του λ,
- αν η επιλογή Β αρχικών μεταβάσεων σε κάθε ζεύγος ως transient period φαίνεται επαρκής ή αν θα προτείνατε διαφορετική τιμή,
- να συγκρίνετε τα αποτελέσματα που έχετε πάρει για τις διαφορετικές τιμές των Τ και Β, ως προς τη σύγκλιση και το χρόνο εκτέλεσης καθώς και τη σχετική διαφορά **με** και **χωρίς** transient period.

- Για να είναι δυνατή η σωστή σύγκριση μεταξύ των διαφορετικών εκτελέσεων, θα πρέπει να χρησιμοποιηθεί η ίδια αρχική τιμή seed σε όλες τις περιπτώσεις.

(4) Τροποποίηση του μοντέλου για ρυθμό εξυπηρέτησης που εξαρτάται από την κατάσταση

- Να περιγράψετε ποιες αλλαγές θα έπρεπε να γίνουν στην προσομοίωσή σας αν ο ρυθμός εξυπηρέτησης δεν ήταν σταθερός, αλλά εξαρτιόταν από την κατάσταση του συστήματος. Συγκεκριμένα, αν στην κατάσταση i ο ρυθμός εξυπηρέτησης ήταν: $\mu_i=\mu(i+1)$ όπου $\mu=1$ πελάτης/sec και $i=1,2,\ldots,10$.

- Να εξηγήσετε πώς θα άλλαζε ο τρόπος παραγωγής των χρόνων εξυπηρέτησης και ποια σημεία του κώδικα θα έπρεπε να τροποποιηθούν.

Η άσκησή μας

Πως θα δουλέψετε

Μοντέλο

M/M/1/10 με έναν εξυπηρετητή, χωρητικότητα 10 και ρυθμό εξυπηρέτησης μ = 5 πελάτες/min

Σενάρια φόρτου

Τρέχετε για λ ∈ {1, 5, 10}
Άρα θα δείτε για λ=1 underloaded
Για λ=5 οριακη κατασταση και
Για λ=10 overloaded λειτουργία

Βασική απαίτηση

Διατηρούμε ίδιο seed σε όλες τις εκτελέσεις, ώστε η σύγκριση μεταξύ διαφορετικών λ να είναι δίκαιη

Μεγέθη που ζητούνται

- Εργοδικές πιθανότητες όλων των καταστάσεων 0,...,10.
- Πιθανότητα απόρριψης πελάτη
- Μέσος αριθμός πελατών στο σύστημα
- Μέσος χρόνος παραμονής στο σύστημα

Ζεύγη εκτέλεσης (T, B)

T μεταβάσεις	B transient
100000	5000
1000000	20000
10000000	120000

Για κάθε ζευγάρι τιμών (T,B) βγάζετε αποτελέσματα δύο φορές: μία με όλες τις μεταβάσεις και μία αφού αγνοήσετε τις πρώτες B ως transient period

Debugging 30 μεταβάσεων

Πρώτα αποδεικνύετε ότι ο simulator δουλεύει σωστά, μετά τρέχετε τα experiments

Trace για τις πρώτες 30 μεταβάσεις

- Τρέχουσα κατάσταση πριν το event
- Είδος επόμενης μετάβασης: άφιξη ή αναχώρηση
- Συνολικός αριθμός αφίξεων μέχρι εκείνη τη στιγμή
- Νέα κατάσταση μετά τη μετάβαση

Κανονικές εκτελέσεις

- Για κάθε $\lambda \in \{1,5,10\}$ τρέχετε και τα 3 ζεύγη (T,B)
- Για κάθε run βγάζετε metrics με όλα τα δεδομένα και χωρίς τις πρώτες B μεταβάσεις
- Χρησιμοποιείτε ακριβώς το ίδιο seed σε όλα τα σενάρια

Μόλις βεβαιωθείτε ότι δεν υπάρχουν παράλογες μεταβάσεις, negative states ή departures από κενό σύστημα, κλείνετε το debug print και περνάτε στις κανονικές εκτελέσεις

Προσοχή

Όχι κατευθείαν γραφήματα χωρίς να έχετε ελέγξει την ορθότητα του event loop

Τι σας ζητείτε να δείξετε στα αποτελέσματα

Γράφημα 1

Τελικές εργοδικές πιθανότητες P0,...,P10. Ιδανικά ένα plot ανά λ ή grouped bars για να φαίνεται η μετατόπιση μάζας

Γράφημα 2

Εξέλιξη της εκτίμησης του μέσου πλήθους L ως προς τον αριθμό μεταβάσεων. **Εκεί φαίνεται αν το transient επηρεάζει πολύ**

Τι ελέγχουμε

Για κάθε λ και (T,B): με transient, χωρίς transient, χρόνος εκτέλεσης, σχετική διαφορά

Σχολιασμός που περιμένουμε

- Πώς αλλάζουν οι εκτιμήσεις όταν κόβετε το αρχικό transient
- Πώς αυξάνει το blocking και το μέσο πλήθος καθώς ανεβαίνει το λ
- Πώς επηρεάζει το T τη σύγκλιση και τη σταθερότητα των εκτιμήσεων

λ=1 έχουμε κανονική λειτουργία και μικρό blocking
λ=5 είναι οριακά κοντά στο capacity pressure
λ=10 το σύστημα κολλάει συχνά κοντά στο 10 και οι απώλειες γίνονται απαραίτητες

Πώς αλλάζει το μοντέλο όταν ο ρυθμός εξυπηρέτησης εξαρτάται από την κατάσταση

Επέκταση: μi = μ(i+1), με μ = 1 πελάτης/sec

Τι αλλάζει

- Ο ρυθμός departure δεν είναι πια σταθερός μ, αλλά εξαρτάται από το τρέχον state i
- Όσο μεγαλώνει το i, το σύστημα “δουλεύει γρηγορότερα” προς την έξοδο
- Άρα ο χρόνος μέχρι την επόμενη αναχώρηση πρέπει να δειγματοληπτείται από Exp(μi)

Τι αλλάζει στον κώδικα

- Στο σημείο όπου υπολογίζεις nextDeparture, βάζεις rate = μ*(n+1) όταν n>0
- Μετά από κάθε άφιξη ή αναχώρηση που αλλάζει το n, ξαναϋπολογίζεις τον departure clock με βάση το νέο state

ΠΡΟΣΟΜΟΙΩΣΗ ΟΥΡΑΣ M/M/1/10 - Ψευδοκώδικας

RANDOM: Ομοιόμορφος τυχαίος αριθμός (0,1)

THRESHOLD: $\lambda/(\lambda+\mu)$

ARRIVALS: Συνολικός αριθμός αφίξεων

ARRIVAL[STATE]: Αριθμός αφίξεων στην κατάσταση STATE = 0, 1, …,10

COUNT: Αριθμός μεταβάσεωνCOUNT = 0, 1, … , MAXIMUM

STATE: Κατάσταση ουράς(πληθυσμός συστήματος M/M/1/10) ,STATE = 0, 1, …, 10

P[STATE]:Εργοδικήπιθανότητα STATE = 0, 1, …, 10

AVERAGE: Μέσος πληθυσμός συστήματος M/M/1/10

INITIALIZE:COUNT = 0, STATE = 0, ARRIVALS = 0, ARRIVAL[0…10] = 0, P[0…10] =0

ARRIVAL:ARRIVALS = ARRIVALS + 1

ARRIVAL[STATE] = ARRIVAL[STATE] + 1

COUNT = COUNT +1

IFSTATE = 10 :***GO TO***LOOP

ELSE: STATE = STATE + 1

GO TOLOOP

LOOP:**IF**STATE = 0 : ***GO TO***ARRIVAL

ELSE :

IFRANDOM < THRESHOLD: ***GO TO***ARRIVAL

ELSE: GO TO DEPARTURE

DEPARTURE :COUNT = COUNT +1 ; STATE = STATE –1

IFCOUNT < MAXIMUM : ***GO TO***LOOP

ELSE: P[STATE=1…10] = ARRIVAL[STATE= 1…10] / ARRIVALS

AVERAGE = SUM { STATE ^ P[STATE]}, STATE = [1…10]

Demo3.m

Αρχικά ξεκινά με άδειο σύστημα

σε κάθε βήμα παράγει το χρόνο μέχρι το επόμενο event

αποφασίζει αν το event είναι arrival ή departure

αν είναι arrival και το σύστημα έχει χώρο, βάζει τον πελάτη

αν είναι arrival και το σύστημα είναι full, τον απορρίπτει

αν είναι departure, αφαιρεί τον παλιότερο πελάτη και μετρά τον χρόνο παραμονής του

ενημερώνει time-average statistics

στο τέλος βγάζει P(i), P_rej, L, W με και χωρίς transient

Περιπτώσεις transient

- **Με transient**: κρατάμε **όλες** τις μεταβάσεις από την αρχή της προσομοίωσης
- **Χωρίς transient**: **πετάμε τις πρώτες Β μεταβάσεις** και μετράμε μόνο μετά από αυτές
  - Αυτό βοηθάει να δούμε την επίδραση της αρχικής μεταβατικής περιόδου

Demo3.m

Setup και counters

lambda = 5, mu = 5, K = 10, T = 10000, B = 1000

state = 0
arrival_attempts, rejections
sum_states, state_counts

Η βασική ιδέα

- Το script κρατά μόνο το πλήθος πελατών state ∈ {0,…,K}.
- Σε κάθε βήμα διαλέγει αν έγινε arrival ή departure

Ο κώδικας προσομοιώνει το πώς αλλάζει η κατάσταση state από μετάβαση σε μετάβαση
Μετράει “πόσες φορές είδα το state i” και όχι “πόσο χρόνο έμεινα στο state i”

Ο βρόχος for step = 1:T

Κάθε iteration αντιστοιχεί σε μία μετάβαση του συστήματος και ενημερώνει counters + νέα κατάσταση

Ροή του loop

1. Μετράμε το τρέχον state στο state_counts και στο sum_states
2. Αν step > B, ενημερώνουμε και τα αντίστοιχα counters _B
3. Αν state = 0, μόνο arrival μπορεί να συμβεί.
4. Αλλιώς: με πιθανότητα lambda/(lambda+mu) έχουμε arrival attempt, αλλιώς departure
5. Αν state = K και έρθει arrival, μετράμε rejection και το state μένει ίδιο.

```
if state == 0
    event = "arrival"; state_new = 1;
else
    if rand() < lambda/(lambda+mu)
        if state < K -> arrival
        else -> rejection
    else
        departure; state_new = state - 1;
    endif
endif
```

- Στο state 0 δεν υπάρχει departure
- Στο state K οι αφίξεις δεν “χάνονται”: μετριούνται ως attempts και μετά ως rejections
- Το debug print των πρώτων 30 βημάτων είναι έλεγχος ορθότητας του loop

Τι υπολογίζουμε

Με όλα τα βήματα

```
P = state_counts / sum(state_counts)

L = sum_states / T

P_rej = rejections / arrival_attempts
```

Αφού κόψουμε το warm-up

```
P_B = state_counts_B /
sum(state_counts_B)

L_B = sum_states_B / (T - B)

P_rej_B = rejections_B /
arrival_attempts_B
```

Επεξηγηματικές σημειώσεις για Κώδικα demo3.m

Ανάλυση Κώδικα

Το demo3.m κάνει τα εξής:

Ορίζει τον seed της γεννήτριας τυχαίων αριθμών

Ορίζει τις βασικές παραμέτρους του συστήματος

Επίσης ορίζει:

το ρυθμό αφίξεων λ
το ρυθμό εξυπηρέτησης μ
τη μέγιστη χωρητικότητα K
τον συνολικό αριθμό μεταβάσεων T
τον αριθμό μεταβάσεων που θα αγνοηθούν στην αρχή B (transient)

Αρχικοποιεί την κατάσταση του συστήματος

Αρχικά το σύστημα θεωρείται άδειο

Άρα:

ο αριθμός πελατών στο σύστημα είναι 0
δεν έχει γίνει ακόμη καμία άφιξη
δεν έχει γίνει καμία απόρριψη
δεν έχει συλλεχθεί ακόμη κανένα στατιστικό

Αρχικοποιεί τους μετρητές για τα στατιστικά

Ανάλυση Κώδικα

Ελέγχει:
πόσες φορές βρέθηκε το σύστημα σε κάθε κατάσταση
πόσες συνολικές προσπάθειες άφιξης έγιναν
πόσες από αυτές απορρίφθηκαν
το άθροισμα των πελατών στο σύστημα σε όλα τα βήματα

Κάνει το ίδιο και για την περίπτωση όπου αγνοούνται οι πρώτες B μεταβάσεις
Ξεκινά την επανάληψη της προσομοίωσης
Ο κώδικας εκτελεί έναν βρόχο από το 1 έως το T

Κάθε επανάληψη αντιστοιχεί σε ένα γεγονός / μία μετάβαση του συστήματος

Καταγράφει την τρέχουσα κατάσταση πριν συμβεί το νέο γεγονός

Στην αρχή κάθε βήματος, ο κώδικας:

μετρά σε ποια κατάσταση βρίσκεται τώρα το σύστημα
ενημερώνει τον πίνακα επισκέψεων καταστάσεων
προσθέτει την τρέχουσα κατάσταση στο άθροισμα που θα χρησιμοποιηθεί για τον μέσο αριθμό πελατών

αν το βήμα είναι μεγαλύτερο από B, ενημερώνει και τους αντίστοιχους μετρητές “μετά το transient”

Εξετάζει αν το σύστημα είναι άδειο

Αν το σύστημα βρίσκεται στην κατάσταση 0:
δεν μπορεί να συμβεί αναχώρηση
άρα το επόμενο γεγονός είναι υποχρεωτικά άφιξη

Σε αυτή την περίπτωση:
αυξάνεται ο μετρητής προσπαθειών άφιξης
ο πελάτης γίνεται δεκτός
η κατάσταση αυξάνεται από 0 σε 1

Αν το σύστημα δεν είναι άδειο, επιλέγει τυχαία το επόμενο γεγονός

Αν το σύστημα δεν είναι άδειο, ο κώδικας παράγει έναν τυχαίο αριθμό και αποφασίζει αν το επόμενο γεγονός θα είναι:
άφιξη, με πιθανότητα ανάλογη του λ
αναχώρηση, με πιθανότητα ανάλογη του μ

Στην απλή αυτή εκδοχή, η απόφαση βασίζεται στο threshold λ/λ+μ

Ανάλυση Κώδικα

Αν επιλεγεί άφιξη, ελέγχει αν υπάρχει διαθέσιμη θέση

Όταν προκύψει γεγονός άφιξης:

αυξάνεται ο συνολικός αριθμός προσπαθειών άφιξης

Στη συνέχεια ο κώδικας ελέγχει:

Αν το σύστημα δεν είναι γεμάτο η άφιξη γίνεται δεκτή και ο αριθμός πελατών αυξάνεται κατά 1

Ανάλυση Κώδικα

Αν το σύστημα είναι γεμάτο η άφιξη απορρίπτεται, αυξάνεται ο μετρητής απορρίψεων και η κατάσταση του συστήματος δεν αλλάζει

Αν το βήμα είναι μετά το B, ενημερώνονται και οι αντίστοιχοι μετρητές μετά το transient

Αν επιλεγεί αναχώρηση, μειώνει την κατάσταση ο αριθμός πελατών μειώνεται κατά 1
Άρα η κατάσταση αλλάζει από state σε state - 1

Εμφανίζει τις πρώτες μεταβάσεις για έλεγχο

Για τα πρώτα 30 βήματα, ο κώδικας τυπώνει στην οθόνη:

τον αριθμό του βήματος
την τρέχουσα κατάσταση
το γεγονός που συνέβη
τη νέα κατάσταση

Ενημερώνει την κατάσταση του συστήματος

Μετά την επιλογή του γεγονότος, ο κώδικας αποθηκεύει τη νέα κατάσταση ώστε να χρησιμοποιηθεί στο επόμενο βήμα

Ανάλυση Κώδικα

Επαναλαμβάνει την ίδια διαδικασία μέχρι να ολοκληρωθούν όλες οι μεταβάσεις

Η ίδια λογική συνεχίζεται μέχρι να εκτελεστούν και τα Τ βήματα της προσομοίωσης

Υπολογίζει τις εργοδικές πιθανότητες των καταστάσεων

Υπολογίζει την πιθανότητα απόρριψης

Η πιθανότητα απόρριψης υπολογίζεται:

μία φορά με όλα τα δεδομένα
και μία φορά αγνοώντας τις πρώτες Β μεταβάσεις

Υπολογίζει τον μέσο αριθμό πελατών στο σύστημα. Υπολογίζεται επίσης και για την περίπτωση μετά το Β.